README — устранение ошибки Helm Chart.yaml при деплое PostgreSQL через GitLab CI → werf → Argo CD

Сценарий: backend уже собирается из Dockerfile через werf, образ публикуется в registry, конфигурация приходит из Vault, приложение разворачивается Argo CD. Для backend нужно добавить PostgreSQL. После добавления `.helm/Chart.yaml` и настройки зависимости CI-проверки проходят, но создание/обновление Argo CD Application завершается ошибкой:

```
level=fatal msg="invalid argument" desc="application spec for <service-name>-backend-dev i
InvalidSpecError: Unable to generate manifests in ...:
rpc error: error getting helm repos:
error retrieving helm dependency repos:
error reading helm chart from /tmp/.../<service-name>-backend/Chart.yaml:
open .../Chart.yaml: no such file or directory"
```

Статус: это диагностическое руководство с рабочим **примером архитектуры**, а не подтверждение конкретной причины: исходные `.gitlab-ci.yml`, `werf.yaml`, Application, структура Git и OCI bundle не предоставлены. Не применять команды, изменяющие существующие ресурсы/данные, без проверки namespace и имён.

Содержание

1. Что именно означает ошибка
2. Самый короткий диагностический маршрут
3. Как должна работать цепочка GitLab → werf → Argo CD
4. Две принципиально разные конфигурации Argo CD: Git и OCI
5. Все основные причины отсутствующего Chart.yaml
6. Правильное подключение PostgreSQL — три варианта
7. Готовый минимальный локальный PostgreSQL subchart
8. Как связать backend, PostgreSQL и Vault
9. Проверки на этапах CI, registry, Argo CD и Kubernetes
10. Матрица ошибок и исправлений
11. Безопасная процедура внедрения и отката
12. Чек-лист готовности и источники

---

1. Что именно означает ошибка

Разберём её по уровням:

```
Application spec is invalid
└── Argo CD не может сгенерировать desired manifests
    └── repo-server обрабатывает Helm chart / его dependencies
        └── repo-server пытается открыть Chart.yaml по конкретному пути
            └── файла там НЕТ (ENOENT)
```

Это пока НЕ ошибка:

- аутентификации пользователя PostgreSQL;
- подключения backend к PostgreSQL;
- PVC/StorageClass/PostgreSQL image;
- разрешений ServiceAccount приложения в кластере;
- `CrashLoopBackOff` PostgreSQL;
- содержимого `DB_HOST`/`DB_PASSWORD`.

К этим пунктам переходить **после** устранения ошибки генерации манифестов.

`error getting helm repos` — это **обёртка**, указывающая на фазу проверки Helm dependencies. Самая информативная часть — `open /tmp/.../Chart.yaml: no such file or directory`. Она **не доказывает**, что удалённый Helm repo недоступен: сперва программа вообще не смогла прочитать локальный `Chart.yaml` в своём рабочем каталоге.

Важная тонкость werf

В werf **основной** chart по умолчанию находится в `.helm/` и при определённых условиях может обходиться без явно созданного `Chart.yaml` — werf подставляет метаданные по умолчанию. **Обычный Helm / Argo CD, читающий chart из Git, так работать не обязан:** ему нужен корректный `Chart.yaml` в корне выбранного chart. Если werf публикует OCI bundle, содержимое `.helm/` должно оказаться **на верхнем уровне опубликованного Helm chart**, а не под каталогом `.helm/`.

Кроме того, `werf bundle publish` в режиме совместимости с Helm (в актуальных версиях — штатный режим) может **заменять name опубликованного чарта последним сегментом пути OCI repo**. Не следует путать `werf.yaml: project`, `.helm/Chart.yaml: name`, имя OCI repo, `spec.source.chart` и `metadata.name` Application: это разные сущности.

---

2. Самый короткий диагностический маршрут

Выполняй **по порядку**, ничего не переименовывая наугад.

Шаг 1. Узнать, откуда Argo CD фактически получает chart

Если есть kubectl и доступ к namespace Argo CD:

```
export APP='<service-name>-backend-dev'
export ARGO_NS='argocd'              # если Argo CD стоит в другом NS — поменять

kubectl -n "$ARGO_NS" get applications.argoproj.io "$APP" -o yaml
```

Если CLI `argocd` установлен:

```
argocd app get "$APP" -o yaml
```

Смотри строго на:

```
spec:
  source:     # или sources: при multi-source
    repoURL: ...
    path: ...
    chart: ...
    targetRevision: ...
    helm: ...
```

Если Application **не существует**, потому что операция его создания завершилась `InvalidSpecError`, возьми манифест/шаблон Application из **GitLab CI, GitOps-репозитория, ApplicationSet или логов команды `argocd app create/set`**. Команда `kubectl get application` в таком случае ничего не найдёт — это ожидаемо.

Вопрос, который нужно разрешить первым:

Наблюдение	Что читает Argo CD	Следующее действие
`repoURL: https://git...`, `path: .helm`	Git-репозиторий	Проверить `.helm/Chart.yaml` **в ревизии `targetRevision`**
`repoURL: https://git...`, `path: .`	Корень Git-репозитория	Проверить `./Chart.yaml`, либо исправить path на `.helm`

Наблюдение	Что читает Argo CD	Следующее действие
`repoURL: registry..., chart: backend`	Helm chart в OCI/Helm repo	Скачать **точно тот же chart/version**, что видит Argo
`repoURL: oci://registry..., path: .`	OCI artifact как source (при поддержке вашей версией Argo CD)	Проверить содержимое OCI artifact и выбранный path
`sources: [...]`	Несколько источников	Проверить **каждый** source, особенно source с path

Шаг 2. Проверить правильный файл в Git

В корне проекта:

```
pwd
ls -la
find . -maxdepth 5 -type f \( -name 'Chart.yaml' -o -name 'chart.yaml' \) -print
ls -la .helm
sed -n '1,100p' .helm/Chart.yaml
git ls-files '.helm/*' '.helm/**' | grep -E 'Chart\.yaml|values\.yaml|charts/'
git status --short
```

Ожидаемое имя: **`Chart.yaml`** (заглавная латинская C), не `chart.yaml`, не `Chart.yml`, не `Charts.yaml`. Linux в GitLab Runner и repo-server чувствителен к регистру.

Для исправления регистра Git при необходимости:

```
git mv .helm/chart.yaml .helm/Chart.yaml
# Если ОС/Git не распознаёт смену только регистра:
# git mv .helm/chart.yaml .helm/chart-tmp.yaml
# git mv .helm/chart-tmp.yaml .helm/Chart.yaml

git add .helm/Chart.yaml
git diff --cached --name-status
```

Затем **закоммить** и убедись, что pipeline использует **новый commit**, а не другой branch/tag.

Шаг 3. Проверить, не смотрит ли Argo CD на старую версию

```
# Когда source Git:
git rev-parse HEAD
git show HEAD:.helm/Chart.yaml

# В CI:
printf 'branch=%s sha=%s pipeline=%s\n' \
  "$CI_COMMIT_REF_NAME" "$CI_COMMIT_SHA" "$CI_PIPELINE_ID"
```

Если Argo читает OCI, новый коммит **сам по себе недостаточен**: необходимо убедиться, что опубликован новый OCI chart **и что `targetRevision` указывает именно на его тег**.

Шаг 4. Проверить уже опубликованный OCI chart, а не только исходный `.helm`

Для Helm-compatible OCI chart при установленном Helm 3 и доступе к registry:

```
# Пример. Подставь ТОЧНЫЕ значения из CI и Application.
export BUNDLE_REPO='registry.example.com/group/bundles/backend'
export CHART_VERSION='1.0.123'       # версия/тег, выбранный в Argo

# При закрытом registry предварительно нужен helm registry login.
helm show chart "oci://${BUNDLE_REPO}" --version "$CHART_VERSION"
```

```
mkdir -p /tmp/backend-chart-check
helm pull "oci://${BUNDLE_REPO}" \
  --version "$CHART_VERSION" \
  --untar --untardir /tmp/backend-chart-check

find /tmp/backend-chart-check -maxdepth 4 -type f \
  \( -name Chart.yaml -o -name values.yaml \) -print
find /tmp/backend-chart-check -maxdepth 5 -type d -name charts -print
```

Вывод проверки:

- `Chart.yaml` в скачанном bundle есть, но Argo ищет другой путь → **ошибка source/path, выбранной версии или интерпретации источника**.
- `Chart.yaml` в bundle отсутствует или он некорректно распакован → **проблема публикации/формата**. Сравнить CLI werf, helm-compatible режим и registry artifact.
- Нужного тега в registry нет → **ошибка версии/публикации**.
- OCI доступен в GitLab job, но не Argo → отдельно проверить **доступ repo-server к registry**, не `imagePullSecrets` workload.

Не делай `helm install` поверх уже управляемых Argo CD ресурсов ради проверки. Для проверки достаточно `helm template` и `helm show/pull`.

---

3. Как должна работать цепочка GitLab → werf → Argo CD

```
GitLab repository
├── werf.yaml
├── Dockerfile / код
└── .helm/
    ├── Chart.yaml
    ├── values.yaml
    ├── templates/              # backend Deployment, Service, ConfigMap
    └── charts/                 # PostgreSQL dependency (если выбран этот подход)
          ↓
GitLab CI: werf build + publish images
          ↓
GitLab CI: werf bundle publish --tag <VERSION>
          ↓
Registry: OCI chart/bundle версии <VERSION>
          ↓
Argo CD Application → source/chart/targetRevision
          ↓
argocd-repo-server загружает chart + dependencies
          ↓
helm template → desired manifests
          ↓
argocd-application-controller синхронизирует resources
          ↓
Kubernetes: backend + PostgreSQL + PVC + Service
```

Vault — отдельная плоскость:

- **CI скачал values из Vault**: это подтверждает только работу CI job. Argo CD **не наследует автоматически** environment variables и временные файлы предыдущей GitLab job.
- **CI передал values в `werf bundle publish`**: проверь, что они реально относятся к публикуемому chart; при этом **нельзя допускать публикацию паролей в OCI bundle**.
- **Vault создаёт/синхронизирует Kubernetes Secret** через оператор или отдельный безопасный процесс: chart может использовать `secretKeyRef` без помещения пароля в

values.
- **Argo CD получает values от отдельного source/плагина**: проверь multi-source/CMP конфигурацию отдельно.

Суть: `values.yaml` — это **параметры Helm-шаблонов**, а не инструкция Kubernetes развернуть PostgreSQL. Если в `Chart.yaml` нет корректной зависимости, либо в `templates/` нет нужных объектов, добавление `postgresql:` в values **само по себе ничего не создаёт**.

---

4. Две принципиально разные конфигурации Argo CD: Git и OCI

Вариант A. Argo CD читает chart напрямую из Git

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: backend-dev
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://gitlab.example.com/group/backend.git
    targetRevision: dev             # либо конкретный commit SHA
    path: .helm                     # <-- ВАЖНО: здесь лежит Chart.yaml
    helm:
      releaseName: backend-dev
      valueFiles:
        - values.yaml
  destination:
    server: https://kubernetes.default.svc
    namespace: backend-dev
```

Когда подходит: Argo CD читает Git напрямую; пайплайн отдельно публикует Docker image, а Argo получает актуальные tags через GitOps/values механизмы.

Типичная ошибка: `path: .` при `Chart.yaml` в `.helm/`. Или `path: postgres`, где chart неполный/не существует.

Важно: для данного варианта Argo не обязан использовать результат `werf bundle publish`. Это **другая схема доставки**, и без обновления values/tag image можно получить несогласованные версии образа и манифестов.

Вариант B. Argo CD читает Helm-compatible OCI bundle, собранный werf

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: backend-dev
  namespace: argocd
spec:
  project: default
  source:
    repoURL: registry.example.com/group/bundles
    chart: backend                  # имя опубликованного OCI chart
    targetRevision: 1.0.123         # существующая версия опубликованного chart
    helm:
      releaseName: backend-dev
  destination:
```

```
    server: https://kubernetes.default.svc
    namespace: backend-dev
```

Здесь OCI chart расположен по адресу вроде:

```
registry.example.com/group/bundles/backend:1.0.123
```

Для классического **Helm-OCI** источника Argo CD поле `repoURL` **без `oci://`**, а `chart` задаётся **отдельно**. Registry должен быть добавлен как OCI-enabled Helm repo с корректными credentials (если приватный).

Типичная ошибка: `chart: backend`, а публикация ушла в `.../service-backend`; либо `targetRevision: 1.0.122`, тогда как pipeline опубликовал `1.0.123`.

Вариант C. Argo CD использует OCI artifact как source

Современные версии Argo CD отдельно поддерживают `repoURL: oci://...` и `path: .` для OCI source:

```
spec:
  source:
    repoURL: oci://registry.example.com/group/bundles/backend
    targetRevision: 1.0.123
    path: .
```

Не путай это с **вариантом B**: синтаксис и регистрация репозитория разные. Поддержку OCI source проверять **по версии установленного Argo CD**. Для более старых версий используй документированную Helm-OCI конфигурацию варианта B.

Если используется `spec.sources` / ApplicationSet

Смотри **все** источники и уже сгенерированный Application:

```
kubectl -n argocd get applicationsets -o wide
kubectl -n argocd get application "$APP" -o yaml
```

Случайный `path` на Git source с одними values способен заставить Argo CD воспринимать его как самостоятельный chart/source. Если источник Git используется **только** как хранилище values для другого chart, обычно требуется `ref: values` **без `path`**, а в Helm source — ссылка `$values/...` (для версий Argo CD с поддержкой multi-source).

---

5. Все основные причины отсутствующего Chart.yaml

5.1. `Chart.yaml` лежит не в том каталоге

Пример:

```
backend/
├── .helm/Chart.yaml            # файл здесь
└── postgres/Chart.yaml         # ещё один чарт здесь
```

Но Argo указывает `path: .` и открывает `backend/Chart.yaml`. **Исправление:** изменить `spec.source.path: .helm` в Git-source, а не переносить всё дерево ради ошибки.

Для **OCI source** наоборот: `.helm/` — это **исходный** каталог werf, а опубликованный chart распаковывается со своим `Chart.yaml` в **корень**. В OCI источнике нельзя вслепую назначать `path: .helm` только потому, что локально используется `.helm/`.

5.2. Неправильный регистр имени файла

Нужно **`Chart.yaml`**. Проверка:

```
find .helm -maxdepth 1 -type f -printf '%f\n'
git ls-tree -r --name-only HEAD | grep -Ei 'chart\.ya?ml$'
```

5.3. Файл не попал в Git, оказался ignored или создан в другой CI job

```
git check-ignore -v .helm/Chart.yaml || true
git ls-files --error-unmatch .helm/Chart.yaml
git show "$CI_COMMIT_SHA":.helm/Chart.yaml
```

Если chart генерируется в одной job, в другой job его **нет автоматически**. Нужны либо **tracked файлы**, либо явная передача через `artifacts`/`needs` с правильным `artifacts: true`. Проверяй `pwd` и `find` **в job, выполняющей `werf bundle publish`**, а не только в validation job.

5.4. Ранее dependency собирали в `postgres/`, а не в основном `.helm`

Плохая схема для единого bundle:

```
backend/
├── .helm/templates/deployment.yaml
└── postgres/
    ├── Chart.yaml
    ├── values.yaml
    └── charts/...
```

Если основной chart **не объявляет** эту зависимость и не содержит её в `.helm/charts/`, публикация основного chart **не обязана** брать файлы из `postgres/`.

Выбор исправления:

1. Вынести PostgreSQL в `.helm/charts/postgresql` как vendored chart; **или**
2. объявить remote dependency в `.helm/Chart.yaml` и собрать её; **или**
3. сделать отдельный Argo CD Application для PostgreSQL.

5.5. Рабочий каталог werf не тот

werf по умолчанию ищет **основной chart в `.helm` относительно Git worktree**, если не изменён `deploy.helmChartDir`.

Проверить:

```
cat werf.yaml
grep -Rni 'helmChartDir\|bundle publish\|cd \|working_dir\|git-work-tree' \
  werf.yaml .gitlab-ci.yml 2>/dev/null || true
pwd
ls -la .helm/Chart.yaml
```

Если в `werf.yaml` используется:

```
deploy:
  helmChartDir: another/path
```

то корректировать нужно **эффективный** путь основного чарта, либо конфигурацию werf. Не дублируй `Chart.yaml` по всему репозиторию.

5.6. `.helmignore`, gitignore, filters, symlink или несовместимый layout

Проверить:

```
cat .helm/.helmignore 2>/dev/null || true
cat .gitignore 2>/dev/null || true
find .helm -maxdepth 4 -type l -ls
find .helm -maxdepth 4 -type f -print
```

Файлы, исключённые из packaging, не появятся в bundle. У werf bundle есть **ограниченный список штатно включаемых файлов/каталогов** (например, `Chart.yaml`, `values.yaml`, `templates/`, `charts/`). Произвольная папка внутри `.helm`, скажем `.helm/postgres/`, не является надёжным местом для публикации chart зависимости.

Есть режим `WERF_BUNDLE_SCHEMA_NONSTRICT=1`, но **не следует использовать его как первую попытку починить ошибочную структуру**: в нормальном случае нужные объекты должны находиться в стандартных местах.

5.7. Argo CD смотрит не на ту версию chart

Частые варианты:

- CI опубликовал OCI tag `1.0.124`, Application держит `1.0.123`;
- `targetRevision` — branch в Git, но Application фактически смотрит в OCI;
- был перезаписан `latest`/другой mutable tag, а repo-server использует кэш;
- pipeline на новой ветке, а Application на старом коммите;
- в Helm-совместимом OCI source неверно определён `chart`.

Решение: тестировать **точный `repoURL` + `chart` + `targetRevision`**, явно обновить Application на новый **уникальный version tag**, затем сделать Refresh/Hard Refresh при необходимости. Перезапуск repo-server — не первое средство.

5.8. `Chart.yaml` существует, но сломан или не соответствует фактическим зависимостям

Проверка:

```
helm lint .helm
helm dependency list .helm
helm template backend-dev .helm -n backend-dev > /tmp/backend-manifests.yaml
```

Пример корректного основного файла:

```
# .helm/Chart.yaml
apiVersion: v2
name: backend
version: 1.0.0
type: application
```

Если dependency remote, дополняется секция `dependencies` (см. далее). Если dependency локальный, у него должен быть собственный корректный `Chart.yaml` и подходящий layout.

5.9. Вложенная `postgresql` dependency ссылается на путь за пределами bundle

Если написать:

```
dependencies:
  - name: postgresql
    version: 0.1.0
    repository: file://../postgres
```

то относительно `.helm` путь может находиться **снаружи** экспортируемого bundle. В Git это может работать, а в распакованном OCI chart — уже нет.

Надёжнее: хранить зависимость внутри `.helm/charts/postgresql/` или как `.tgz` внутри `.helm/charts/`; либо использовать **доступный всем сторонам** OCI/HTTP chart repository. Использование `file://` допустимо, если путь гарантированно существует **в окружении,**

где выполняется dependency resolution; не считать локальный успех доказательством работоспособности в Argo CD.

5.10. Смешаны две модели: Git chart и OCI bundle

Признак: публикация `werf bundle publish` проходит, но `spec.source.repoURL` указывает на Git и `path: .`, либо наоборот — Git изменили, но Argo остаётся на старом OCI artifact.

Решение: выбрать единственный **источник истины** для деплоя и проследить, как обновляется его revision. Само наличие двух успешных pipeline jobs связи между ними не создаёт.

---

6. Правильное подключение PostgreSQL — три варианта

Вариант 1 — remote chart dependency в основном `.helm/Chart.yaml`

```yaml
# .helm/Chart.yaml
apiVersion: v2
name: backend
version: 1.0.0
type: application

dependencies:
  - name: postgresql
    version: '<PINNED_EXISTING_CHART_VERSION>'
    repository: '<REAL_HTTP_OR_OCI_CHART_REPOSITORY>'
    condition: postgresql.enabled
```

Значения для PostgreSQL должны задаваться **под ключом `postgresql`**, но **точные имена параметров** (`auth`, `primary`, `persistence`, `tls` и т. д.) зависят от **конкретного внешнего чарта и его версии**. Нельзя переносить значения из одного PostgreSQL chart в другой без проверки `helm show values`.

```
# При добавлении/изменении remote зависимости:
werf helm dependency update .helm
# Сохранить корректный Chart.lock, проверить charts/:
werf helm dependency build .helm
helm dependency list .helm
helm lint .helm
```

Для Helm CLI также доступны `helm dependency update/build .helm`. `dependency update` обновляет lock, `dependency build` воспроизводит версии из lock (если lock есть).

Нюансы:

- фиксируй проверенную версию chart, не «последнюю»;
- проверь существование репозитория и доступ из Argo `repo-server`;
- OCI dependency может требовать **отдельной** авторизации;
- если `.helm/charts/*.tgz` не включены в Git, проверь, собираются ли они при публикации bundle и реально входят ли в **опубликованный chart**;
- передача секретов через chart values требует отдельной модели защиты;
- в 2026 году у ряда поставщиков образов/чартов изменялись условия доступа: проверяй конкретный chart и image перед внедрением, не делай предположений о публичной доступности.

Подходит: нужен готовый сопровождаемый chart; есть контроль версий и сетевого доступа.

Вариант 2 — локальный PostgreSQL subchart в `.helm/charts/postgresql/`

```
backend/
├── werf.yaml
├── .gitlab-ci.yml
└── .helm/
    ├── Chart.yaml                # ОБЯЗАТЕЛЬНО для обычного Helm consumer
    ├── values.yaml
    ├── templates/
    │   ├── backend-deployment.yaml
    │   ├── backend-service.yaml
    │   └── backend-configmap.yaml
    └── charts/
        └── postgresql/
            ├── Chart.yaml        # ОБЯЗАТЕЛЬНО для subchart
            └── templates/
                ├── service.yaml
                └── statefulset.yaml
```

Плюсы: нет зависимости от внешнего Helm repo на этапе деплоя; структура bundle прозрачна. **Минусы:** за сопровождение PostgreSQL chart, security, backup, обновления и восстановление отвечаешь ты. Ниже есть пример для dev/test.

Вариант 3 — отдельный Argo CD Application для БД

```
Argo Application: backend-dev
  └── Backend Deployment/Service/ConfigMap

Argo Application: backend-postgresql-dev
  └── PostgreSQL StatefulSet/Service/PVC/Secret references
```

Часто лучший операционный вариант для долгоживущей базы: отдельный lifecycle, синхронизация, storage, backup/restore и правила prune. Для серьёзного production стоит оценить PostgreSQL Operator (например, CloudNativePG) и внешнюю управляемую БД.

Важно: если хочешь PostgreSQL **в том же namespace**, это совместимо с отдельным Application: `destination.namespace` может совпадать. Отдельный Application не означает отдельный namespace.

---

7. Готовый минимальный локальный PostgreSQL subchart

> **Назначение примера — показать полностью корректное подключение подчинённого чарта в dev/test.** Это **не production-ready PostgreSQL**: здесь нет настроенных WAL/PITR backup, TLS server certs, HA/replication, планового восстановления и продуманного механизма major upgrades. Пример **не удаляет** и **не мигрирует** существующую БД. PostgreSQL запускается как **одиночный StatefulSet с PVC**.

7.1. Главный chart — `.helm/Chart.yaml`

```yaml
apiVersion: v2
name: backend
version: 1.0.0
type: application

dependencies:
  - name: postgresql
    version: 0.1.0
```

```
    condition: postgresql.enabled
    # repository отсутствует: chart физически лежит в .helm/charts/postgresql
```

У werf поддерживаются локальные зависимости из `.helm/charts/`. Если по особенностям версии Helm dependency management у тебя появляются ошибки на `helm dependency build`, **не пытайся скачивать локальный chart как remote**: проверь, что `postgresql/Chart.yaml` доступен и имя/version совпадают; для vendored chart, уже лежащего в `charts/`, повторное скачивание не требуется. Команда `helm template` — конечная проверка.

7.2. Values родительского chart — `.helm/values.yaml`

```yaml
postgresql:
  enabled: true
  image:
    repository: postgres
    tag: '16-alpine'              # закрепи digest для production
  auth:
    username: appuser
    database: appdb
    existingSecret: backend-pg-auth
    passwordSecretKey: password
  persistence:
    size: 20Gi
  resources:
    requests:
      cpu: 100m
      memory: 256Mi
    limits:
      memory: 1Gi
```

Этот файл не содержит пароль. `backend-pg-auth` должен существовать **в целевом namespace** до запуска контейнера; он синхронизируется из Vault отдельно.

7.3. Зависимый chart — `.helm/charts/postgresql/Chart.yaml`

```yaml
apiVersion: v2
name: postgresql
version: 0.1.0
type: application
appVersion: '16'
```

7.4. Сервисы — `.helm/charts/postgresql/templates/service.yaml`

```yaml
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-postgresql
  labels:
    app.kubernetes.io/name: postgresql
    app.kubernetes.io/instance: {{ .Release.Name }}
spec:
  type: ClusterIP
  selector:
    app.kubernetes.io/name: postgresql
    app.kubernetes.io/instance: {{ .Release.Name }}
  ports:
    - name: postgresql
      port: 5432
```

```
      targetPort: postgresql
---
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-postgresql-headless
  labels:
    app.kubernetes.io/name: postgresql
    app.kubernetes.io/instance: {{ .Release.Name }}
spec:
  clusterIP: None
  selector:
    app.kubernetes.io/name: postgresql
    app.kubernetes.io/instance: {{ .Release.Name }}
  ports:
    - name: postgresql
      port: 5432
      targetPort: postgresql
```

7.5. StatefulSet — `.helm/charts/postgresql/templates/statefulset.yaml`

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: {{ .Release.Name }}-postgresql
  labels:
    app.kubernetes.io/name: postgresql
    app.kubernetes.io/instance: {{ .Release.Name }}
spec:
  serviceName: {{ .Release.Name }}-postgresql-headless
  replicas: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: postgresql
      app.kubernetes.io/instance: {{ .Release.Name }}
  template:
    metadata:
      labels:
        app.kubernetes.io/name: postgresql
        app.kubernetes.io/instance: {{ .Release.Name }}
    spec:
      containers:
        - name: postgresql
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          imagePullPolicy: IfNotPresent
          ports:
            - name: postgresql
              containerPort: 5432
              protocol: TCP
          env:
            - name: POSTGRES_USER
              value: {{ .Values.auth.username | quote }}
            - name: POSTGRES_DB
              value: {{ .Values.auth.database | quote }}
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
```

```
                name: {{ .Values.auth.existingSecret | quote }}
                key: {{ .Values.auth.passwordSecretKey | quote }}
          - name: PGDATA
            value: /var/lib/postgresql/data/pgdata
        volumeMounts:
          - name: postgres-data
            mountPath: /var/lib/postgresql/data
        resources:
          {{- toYaml .Values.resources | nindent 12 }}
        startupProbe:
          exec:
            command:
              - /bin/sh
              - -ec
              - 'pg_isready -U "$POSTGRES_USER" -d "$POSTGRES_DB" -h 127.0.0.1'
          periodSeconds: 5
          failureThreshold: 60
        readinessProbe:
          exec:
            command:
              - /bin/sh
              - -ec
              - 'pg_isready -U "$POSTGRES_USER" -d "$POSTGRES_DB" -h 127.0.0.1'
          periodSeconds: 5
          failureThreshold: 6
        livenessProbe:
          exec:
            command:
              - /bin/sh
              - -ec
              - 'pg_isready -U "$POSTGRES_USER" -d "$POSTGRES_DB" -h 127.0.0.1'
          periodSeconds: 10
          failureThreshold: 6
  volumeClaimTemplates:
    - metadata:
        name: postgres-data
      spec:
        accessModes:
          - ReadWriteOnce
        # Если default StorageClass нет, добавь проверенный storageClassName.
        resources:
          requests:
            storage: {{ .Values.persistence.size | quote }}
```

Примечания к StatefulSet:

- Если в кластере **нет default StorageClass**, PVC останется Pending. Тогда добавь `storageClassName` под свою storage-систему.
- Образ `postgres:16-alpine` требует проверки совместимости с Pod Security Standards; официальный entrypoint может менять владельца data-directory при старте. **Не добавляй `runAsNonRoot/readOnlyRootFilesystem` вслепую** — протестируй с вашей storage и политиками.
- В PGDATA используется вложенная папка pgdata, чтобы не инициализировать базу прямо в корне PVC (на некоторых файловых системах есть `lost+found`).
- `POSTGRES_USER`, `POSTGRES_DB`, `POSTGRES_PASSWORD` **инициализируют только новый пустой data directory**. Изменение этих ENV при существующем PVC **не переименует роль/базу и не сменит пароль уже работающей PostgreSQL**. Для этого нужны SQL/операционная процедура.

- pg_isready показывает готовность принимать соединения, но **не доказывает успешную аутентификацию приложения и не проверяет миграции**.
- StatefulSet и PVC не обеспечивают backup. Snapshot и резервная копия — разные гарантии; отдельно настрой backup и **тест восстановления**.
- Для prod нужны ограничения доступа NetworkPolicy, TLS с проверкой сертификата (в соответствующей threat model), pinning image digest, мониторинг, регулярные restore-тесты, план апгрейдов.

7.6. Проверить генерацию манифестов на машине с Helm

```
helm lint .helm
helm dependency list .helm
helm template backend-dev .helm --namespace backend-dev > /tmp/backend-render.yaml

grep -nE '^kind: (Deployment|Service|StatefulSet)|^  name: backend-dev-postgresql' \
  /tmp/backend-render.yaml
```

Должны появиться: StatefulSet PostgreSQL, **два** Service (обычный и headless), Deployment и Service backend (если шаблоны backend уже были в `.helm/templates`).

Если manifest PostgreSQL отсутствует:

1. `postgresql.enabled` должен быть `true`;
2. имя в `dependencies[].name` = имя папки = `Chart.yaml:name`;
3. subchart не должен быть исключён `.helmignore`;
4. выполняй `helm template` для **родителя `.helm`**, не для одинокого `postgresql`;
5. проверь в локальном и OCI chart, что действительно присутствуют файлы `.helm/charts/postgresql/...` (в опубликованном chart — `charts/postgresql/...`).

---

8. Как связать backend, PostgreSQL и Vault

8.1. Где должны быть секреты

Рекомендуемая цепочка:

```
Vault: secret/data/dev/backend/postgresql
    ↓
External Secrets Operator / другой разрешённый механизм
    ↓
Kubernetes Secret: backend-dev/backend-pg-auth
    └── key: password
         ↓
PostgreSQL StatefulSet: POSTGRES_PASSWORD <- secretKeyRef
Backend Deployment: DB_PASSWORD <- secretKeyRef
```

Наименование `backend-pg-auth` — **пример**, а не команда создать секрет в существующем namespace.

Важно: один и тот же secret должен хранить пароль, который фактически настроен для роли `appuser` в PostgreSQL. После инициализации данных смена Vault secret не обязательно меняет пароль **внутри БД** (см. пункт про PGDATA).

Проверить без раскрытия пароля:

```
export NS='backend-dev'

kubectl -n "$NS" get secret backend-pg-auth
kubectl -n "$NS" get secret backend-pg-auth -o jsonpath='{.data.password}' | wc -c
```

Вторая команда **не выводит** пароль, а показывает количество байтов его base64-представления. Не сохраняй вывод `kubectl get secret -o yaml`, `helm get values`, `argocd app manifests` и CI stdout в публичных логах, если там могут быть секреты.

8.2. Пример фрагмента backend Deployment

Это **только фрагмент блока `env`** внутри уже существующего `.helm/templates/deployment.yaml`; **не добавляй второй Deployment**, если он у тебя уже настроен:

```yaml
env:
  - name: DB_HOST
    value: "{{ .Release.Name }}-postgresql"
  - name: DB_PORT
    value: "5432"
  - name: DB_NAME
    value: {{ .Values.postgresql.auth.database | quote }}
  - name: DB_USER
    value: {{ .Values.postgresql.auth.username | quote }}
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: {{ .Values.postgresql.auth.existingSecret | quote }}
        key: {{ .Values.postgresql.auth.passwordSecretKey | quote }}
```

Для твоего backend **замени только имена переменных** на те, которые реально читает приложение (например, `DATABASE_URL`, `PGHOST`, `POSTGRES_*` или другие). Если chart уже подставляет эти параметры через ConfigMap, не нужно дублировать их туда и сюда; пароль всегда из Secret.

Обычно Service DNS внутри одного namespace:

`backend-dev-postgresql:5432`

Для другой namespace:

`backend-dev-postgresql.backend-dev.svc.cluster.local:5432`

Если Argo `helm.releaseName` другое, Service name может быть другим. Проверяй итоговый `helm template`/`kubectl get svc`, не угадывай.

8.3. Особенности Vault, которые легко пропустить

Ситуация	Что происходит	Исправление
CI загрузил секреты в job `load-config`	Они существуют только в этой job	Явная передача нужных несекретных values через artifacts/needs либо Secret-sync механизм
В job `werf bundle publish` передали пароль через `--set`/values	Пароль может попасть в опубликованный chart/его values	**Не включать пароль в OCI/Helm chart**; использовать K8s Secret + `secretKeyRef`
ExternalSecret есть, Ready=False	Vault authentication/path/property failure	`kubectl describe externalsecret`; controller logs; проверить SecretStore
K8s Secret есть, PostgreSQL всё равно не входит	Пароль уже другой внутри PVC	Разобраться с состоянием БД; не стирать PVC
Backend и PostgreSQL в разных NS	Short DNS name не совпадает; secrets namespaced	FQDN Service и Secret в правильном NS

Ситуация	Что происходит	Исправление
PostgreSQL защищён TLS, backend использует `sslmode=disable`	TLS handshake/политики безопасности	Настроить клиент TLS и проверку server CA/hostname; `require` не равно `verify-full`

---

9. Проверки на этапах CI, registry, Argo CD и Kubernetes

9.1. GitLab CI — добавить диагностическую preflight job

Подстрой stage и container image под текущий pipeline. **Не заменяй существующую сборку werf на этот пример**:

```yaml
helm-preflight:
  stage: validate
  image:
    name: alpine/helm:3.16.4  # ПРИМЕР, зафиксируй поддерживаемый образ/версию
    entrypoint: [""]
  script:
    - pwd
    - test -f .helm/Chart.yaml
    - test -d .helm/templates
    - helm lint .helm
    - helm dependency list .helm
    - helm template backend-dev .helm --namespace backend-dev > /tmp/rendered.yaml
    - grep -q 'kind: StatefulSet' /tmp/rendered.yaml
    - grep -q 'kind: Service' /tmp/rendered.yaml
```

Здесь `grep -q 'kind: StatefulSet'` предполагает PostgreSQL как StatefulSet из примера. Если выбрал **готовый внешний chart с другим набором ресурсов**, проверяй соответствующие ресурсы. Эта job — **дополнительный guard**, а не универсальный тест всех конфигураций.

Для remote dependencies: сначала собрать зависимости в job/образе, имеющем доступ к repo, а затем сделать lint/template. При использовании отдельной job artifacts необходимо явно передать в preflight. Не использовать непроверенный tag `alpine/helm` в production CI без pinning digest и проверки supply-chain.

9.2. В job `werf bundle publish` логировать структуру ПЕРЕД публикацией

```bash
set -eu
printf 'PWD=%s\n' "$PWD"
printf 'CI_COMMIT_SHA=%s\n' "${CI_COMMIT_SHA:-unknown}"
ls -la .helm
find .helm -maxdepth 4 -type f | sort
sed -n '1,100p' .helm/Chart.yaml

# Далее ваш существующий werf bundle publish, с версией/тегом
```

Не логируй values, если в файлах бывают пароли. `find` показывает имена файлов; перед выводом оцени, нет ли чувствительной информации даже в путях.

9.3. После публикации открыть ТОТ ЖЕ chart через registry

```bash
# В защищённой job, где доступны credentials registry:
export REGISTRY='registry.example.com'
export BUNDLE_REPO='registry.example.com/group/bundles/backend'
export BUNDLE_VERSION='1.0.123'
```

```
# Пример для GitLab CI variables (не печатай password):
printf '%s' "$CI_REGISTRY_PASSWORD" | helm registry login "$REGISTRY" \
  --username "$CI_REGISTRY_USER" --password-stdin

helm show chart "oci://${BUNDLE_REPO}" --version "$BUNDLE_VERSION"
helm pull "oci://${BUNDLE_REPO}" --version "$BUNDLE_VERSION" \
  --untar --untardir /tmp/published

find /tmp/published -name Chart.yaml -o -name '*.tgz'
# Проверьте manifest именно опубликованного chart:
CHART_DIR="$(find /tmp/published -mindepth 1 -maxdepth 1 -type d | head -n1)"
test -n "$CHART_DIR"
helm template backend-dev "$CHART_DIR" -n backend-dev > /tmp/published-render.yaml
```

Ожидается: одинаковый набор backend и PostgreSQL объектов в **локально отрендеренном** и **опубликованном** chart. Допустимы отличия из-за werf image values/annotations или других явно переданных values, но **PostgreSQL не должен исчезнуть**.

Если у вас **старый/специальный формат bundle, несовместимый с обычным `helm pull`**, используй соответствующие `werf bundle render`/`werf bundle copy` из установленной версии werf. Сам факт, что registry содержит OCI artifact, не гарантирует, что это стандартный Helm-compatible chart.

9.4. Argo CD — status, repo-server и ApplicationSet

```
# Подставить реальный NS установки Argo CD:
export ARGO_NS='argocd'
export APP='<service-name>-backend-dev'

kubectl -n "$ARGO_NS" get application "$APP" -o yaml
kubectl -n "$ARGO_NS" get pods -l app.kubernetes.io/name=argocd-repo-server -o wide
kubectl -n "$ARGO_NS" logs deployment/argocd-repo-server \
  --all-containers=true --since=30m --tail=300

# Если логов несколько/несколько replica — выбрать нужный pod:
# kubectl -n "$ARGO_NS" logs <repo-server-pod> -c argocd-repo-server --since=30m
```

На UI: **Applications → конкретный backend Application → Details / Source / Conditions / Events**. Ищи repoURL, `chart`, `path`, `targetRevision`, точную ошибку и возможность Refresh/Hard Refresh.

Не начинай с `kubectl logs` backend Pod — если chart не отрендерился, новый PostgreSQL Pod ещё не создан и ничего полезного там не будет.

9.5. Если Chart.yaml найден, но dependency не скачивается

Проверить в **argocd-repo-server**:

- DNS resolution и egress к Helm/OCI repo;
- TCP/TLS handshake, доверие к custom CA;
- credentials репозитория (не путать с kubeconfig и workload imagePullSecret);
- URL и тип repo (`helm`, `oci`), а также версию chart;
- права доступа к registry/tag, rate limits;
- `file://` dependencies: существуют ли локальные каталоги **после скачивания chart**;
- proxy/NetworkPolicy/firewall;
- chart `Chart.lock` соответствует `Chart.yaml`;
- dependencies физически доступны внутри `charts/` либо repo-server имеет возможность их восстановить.

Что покажут ошибки:

- `401 Unauthorized` / `403 Forbidden` → registry credentials/permissions;
- `no such host` → DNS;
- `x509: certificate signed by unknown authority` → CA/certificate trust;
- `connection timeout` → egress/network;
- `chart not found` → repository/name/version;
- `found in Chart.yaml, but missing in charts/` → dependency packaging/download;
- `open .../Chart.yaml: no such file or directory` → **вернуться к разделам 2, 4, 5**, прежде чем проверять credentials.

9.6. Kubernetes — после успешной генерации манифестов

```
export NS='backend-dev'

kubectl -n "$NS" get deploy,sts,svc,pods,pvc -o wide
kubectl -n "$NS" get events --sort-by=.lastTimestamp | tail -n 40
kubectl -n "$NS" describe sts backend-dev-postgresql
kubectl -n "$NS" describe pod backend-dev-postgresql-0
kubectl -n "$NS" logs sts/backend-dev-postgresql -c postgresql --tail=100
kubectl -n "$NS" get svc backend-dev-postgresql
kubectl -n "$NS" get pvc
```

Если backend получает Secret и имеет сетевой доступ, можно проверить DNS/TCP из его контейнера (если там установлены инструменты) либо из отдельного **разрешённого диагностического Pod** в том же namespace. Не публикуй пароль в аргументах `psql -W` или командной строке; используй существующие секреты и интерактивный ввод.

9.7. Диагностика PVC и PostgreSQL

Симптом	Проверка	Действие
`PVC Pending`	`kubectl describe pvc`	default StorageClass / capacity / topology / provisioner
`FailedMount`	`describe pod`, CSI logs	Storage/permissions/volume attachment
`permission denied` в PGDATA	`kubectl logs` + securityContext/storage	Исправить UID/GID/volume ownership, не ослаблять безопасность вслепую
`database files incompatible`	PG startup logs + version	Не выполнять major upgrade простым изменением image tag; backup + `pg_upgrade`/миграция
`password authentication failed`	PG logs, actual user/db/Secret	Проверить и согласовать реальные role credentials; не удалять PVC
`connection refused`	pod readiness + endpoints + service	PostgreSQL не слушает/не ready, Service или endpoints неверны
`timeout`	DNS/endpoints/NetworkPolicy	Проверить маршрутизацию и ограничения сети
DB пустая или нет таблиц	migrations	Проверить, выполняются ли backend migrations и к правильной ли базе они подключаются

10. Матрица ошибок и исправлений

Приоритет	Ошибка / состояние	Наиболее вероятная причина	Где смотреть / как исправить
P0	`.../backend/Chart.yaml no such file`	Argo смотрит не в chart-root	`spec.source.path` / извлечённый OCI chart
P0	`.helm/Chart.yaml` есть локально, но ошибка та же	Арго смотрит Git root/старую ревизию/другой artifact	`repoURL`, `chart`, `path`, `targetRevision`
P0	Argo `InvalidSpecError` ещё при create	Manifest generation validation падает до sync	Generated Application spec и `argocd-repo-server`
P1	`chart.yaml` вместо `Chart.yaml`	Case sensitivity	`git ls-tree`, переименование через `git mv`
P1	`helm template` locally OK, OCI без PostgreSQL	Не включили subchart/артефакты в bundle	`helm pull` опубликованной версии; CI worktree
P1	`helm template` локально видит только backend	Values — не ресурсы, dependency не подключена	`.helm/Chart.yaml`, `.helm/charts`, condition
P1	`failed to download dependencies`	Недоступен OCI/HTTP repo или неверная версия	Repo-server network/auth/certs, `helm dependency list`
P1	`Error: no repository definition`	Ошибочно сформирован dependency repo / alias	repository field, lock, local vs remote chart
P1	`dependency ... not found in charts`	Пакет не попал в runtime chart	`helm dependency build`, vendored `.tgz`, published artifact
P1	`chart name mismatch`	Не совпадают dependency name, directory, Chart.yaml	Исправить name/version (при OCI ещё chart name)
P2	`Secret not found`	Vault sync не создал Secret или не тот NS	`kubectl get secret`, ESO events
P2	PG Pod Pending	Нет доступного PVC/StorageClass	`kubectl describe pvc`
P2	PG CrashLoopBackOff	permissions, initdb, version, data corruption	Pod events, PG logs
P2	backend не соединяется с DB	host/user/db/password/NetworkPolicy/DNS	Service, actual role, Secret, backend logs
P2	PG развёрнута, но Argo `OutOfSync`	resources drift/ownership/immutable fields	Argo diff, desired vs live manifests
P2	PG есть, но schema отсутствует	Не запускались migrations	Job/init process backend, версии миграций

11. Безопасная процедура внедрения и отката

Порядок изменений

Фаза A — ничего не меняем в кластере:

1. Сохранить исходный Application spec/werf config/CI version.
2. Выяснить тип источника Argo CD: Git или OCI.
3. Определить **точный chart root** и revision.
4. Проверить регистр/отслеживание `Chart.yaml`.
5. Скачать опубликованный chart **того же tag**, что у Application.
6. Устранить отсутствие файла/неверную ссылку на chart.
7. Прогнать `helm lint` + `helm template` + проверку опубликованного artifact.

Фаза B — готовим PostgreSQL:

8. Выбрать remote chart, local subchart либо отдельное Application.
9. Убедиться, что образ/версия PostgreSQL валидны и доступны кластеру.
10. Проверить StorageClass, PVC size, namespace, credentials из Vault.
11. Проверить манифесты: PostgreSQL StatefulSet/Service/PVC + backend env.
12. Настроить backup/restore до записи ценных данных.

Фаза C — деплоим:

13. Опубликовать **уникальную версию** chart.
14. Перевести Argo `targetRevision` на эту версию/commit по принятой GitOps-процедуре.
15. Проверить Manifest generation — должна проходить **до** sync.
16. Провести Argo Sync; проверить PostgreSQL PVC/Pod/Service и доступ backend.
17. Выполнить миграции приложения и smoke test.

Что НЕЛЬЗЯ делать для «быстрого исправления»

- `kubectl delete pvc` или удаление data volume — **потеря данных**.
- Удалять PostgreSQL StatefulSet, не проверив reclaimPolicy/retention и backup.
- Запускать одновременно второй PostgreSQL на том же data directory.
- Менять `postgres:16` на 17/18 и ожидать автоматического major upgrade существующего `PGDATA`.
- Вставлять пароли в `.helm/values.yaml`, `Chart.yaml`, ConfigMap и публикуемый bundle.
- Бездумно выключать TLS verification или давать Argo админские права.
- Перезапускать весь Argo CD для ошибки неправильного path/`Chart.yaml`.
- Менять mutable `latest` и объявлять успешность без сравнения artifact digest/revision.
- Делать `helm install` той же БД «параллельно» с Argo CD.
- Включать автоматический prune, пока не проверено, какие **существующие** resources перестанут входить в chart; для БД это особенно критично.

Если исправление не помогло

Отладка **по принципу сравнения двух источников**:

```
1. Git HEAD                        → Есть .helm/Chart.yaml?
2. CI worktree в publish job       → Есть .helm/Chart.yaml + subchart?
3. OCI chart того же tag           → Есть Chart.yaml + charts/postgresql?
4. Argo Application source         → Тот ли repo/chart/path/targetRevision?
5. Argo repo-server logs           → Из какого точного /tmp пути читает?
6. helm template опубликованного → Есть StatefulSet/Service PG?
7. Live Kubernetes                → Есть PG Pod/PVC/Service, Ready ли они?
```

Найди **первый шаг, где ожидаемый файл/ресурс исчезает**. Именно между этим шагом и предыдущим следует исправлять проблему — а не в каждом звене сразу.

План отката

- Если Manifest generation не прошёл: **не нужно трогать PVC**, верни Application к последней рабочей версии/source.
- Если появился PG StatefulSet с PVC: откатывать **только манифесты с учётом жизненного цикла PVC**; не рассчитывай, что Argo автоматически сохранит всё нужное при prune.
- Если выполнены DB migrations: откат backend chart **не означает** откат схемы БД. Необходимы совместимые миграции либо заранее протестированный restore.
- Если был изменён пароль: восстанавливай согласованность Secret и PostgreSQL роли отдельной процедурой.

---

12. Чек-лист готовности и источники

Минимум, при котором проблема `Chart.yaml` считается устранённой

- ☐ Известен **реальный** тип Argo source: Git/Helm-OCI/OCI artifact.
- ☐ Известны его `repoURL`, `path` или `chart`, `targetRevision`.
- ☐ `Chart.yaml` находится в корне **выбранного** chart.
- ☐ Git/CI/OCI содержат корректный chart **той самой ревизии**.
- ☐ Все зависимости доступны и включены в рендеринг.
- ☐ `helm template` опубликованного chart создаёт PostgreSQL resources.
- ☐ Argo больше не возвращает `InvalidSpecError` при генерации манифестов.

Минимум, при котором PostgreSQL считается подключённой

- ☐ PostgreSQL Pod Ready.
- ☐ PVC Bound и место адекватно.
- ☐ Service с правильным именем/портом существует.
- ☐ Backend получает корректные DB_HOST/PORT/USER/NAME.
- ☐ Пароль приходит только через Secret и соответствует роли в самой БД.
- ☐ Backend может установить соединение и выполнить тестовый запрос.
- ☐ Миграции приложения успешно отработали.
- ☐ Backup/restore запланированы и проверены перед продуктивными данными.

Что собрать, если нужны будут точечные правки конкретно вашего проекта

Без паролей, токенов, содержимого K8s Secret и приватных Vault путей:

1. `tree -a -L 4 или find .helm -maxdepth 4 -type f`
2. `.helm/Chart.yaml`
3. `.helm/values.yaml (с замаскированными секретами)`
4. `.helm/charts/postgresql/Chart.yaml или внешний dependency declaration`
5. `werf.yaml (часть project/configVersion/deploy)`
6. `.gitlab-ci.yml (werf publish и Argo stage)`
7. `spec.source/spec.sources созданного Application или исходного шаблона`
8. `полный error из Argo repo-server`
9. `версии werf / Helm / Argo CD`
10. `точный published OCI tag и имя репозитория (можно обезличить)`

Проверенные документационные ориентиры

- werf — Charts and dependencies: https://werf.io/docs/latest/usage/deploy/charts.html
- werf — Bundles and charts: https://werf.io/docs/latest/usage/distribute/bundles.html
- werf — `bundle publish`: https://werf.io/docs/latest/reference/cli/werf_bundle_publish.html
- werf — `helm dependency update`: https://werf.io/docs/latest/reference/cli/werf_helm_dependency_update.ht
- Argo CD — Helm integration: https://argo-cd.readthedocs.io/en/latest/user-guide/helm/

- Argo CD — OCI source: https://argo-cd.readthedocs.io/en/latest/user-guide/oci/
- Argo CD — Private repositories / Helm OCI auth: https://argo-cd.readthedocs.io/en/stable/user-guide/private-repositories/
- Argo CD — Multiple sources: https://github.com/argoproj/argo-cd/blob/master/docs/user-guide/multiple_sources.md
- Helm — Dependency management: https://helm.sh/docs/helm/helm_dependency/
- Аналогичное сообщение об отсутствующем `Chart.yaml` в repo-server: https://github.com/argoproj/argo-cd/issues/22124

Краткий итог именно по твоему инциденту

С высокой вероятностью причина найдётся **до этапа подключения к PostgreSQL**: в том, **куда смотрит `spec.source` Argo CD**, либо в различии между **исходным `.helm` и опубликованным через werf chart**.

Первая команда — получить `spec.source` или шаблон создаваемого Application. **Вторая проверка** — скачать точно тот bundle/chart/tag, который указывается там, и открыть его `Chart.yaml`. Только когда обе стороны совпадут, имеет смысл исправлять зависимость PostgreSQL и values.

Успех `werf bundle publish` сам по себе **не гарантирует**, что нужный Argo Application читает именно этот bundle.